— Practical guide for operators

Turn Papers into a Knowledge Graph

Install to upload — with copy-paste code, real outputs, recipes, and troubleshooting. No theory you don't need.

INPUT	plain text extracted from a paper (not a PDF)
PROCESS	Stage 1 facts → Stage 2 frames → Stage 3 ontology mapping
OUTPUT	validated N-Quads — a graph you can load

```
text ›› analyze_paper() ›› results ›› export_neptune_nquads() ›› .nq
```

CONTENTS

In this manual

Everything is task-oriented: find what you want to do, copy the code, run it.

CHAPTER 01

What it does & what you need

You give it the text of a paper. It gives you back structured, verified, source-anchored facts — and, optionally, a knowledge-graph file ready to upload.

› HOW IT WORKS, IN 30 SECONDS

Three stages run automatically for each paragraph:

STAGE	WHAT IT PRODUCES
Stage 1	Atomic **facts**, each labeled (measurement? cause? prediction?) and tied to its exact source text.
Stage 2	Verified **frames** — the facts organized into fields, grounding-checked, with a confidence score.
Stage 3	The facts mapped onto an **ontology** and emitted as RDF for a graph database (optional — only if you give it an ontology).

› WHAT YOU NEED BEFORE YOU START

PYTHON 3.10+

It's a library you import — not an app or a server.

A LANGUAGE MODEL

Your own — an API key (Anthropic/OpenAI) or a local/hosted model. The library brings none.

TEXT, NOT PDFS

Convert PDFs to text first (see Chapter 2). The library reads plain text.

AN ONTOLOGY (STAGE 3 ONLY)

An `.owl` / `.ttl` file for your subject area. Skip it to get just facts.

> **MENTAL MODEL —**
>
> You supply the electricity (your model) and, optionally, the blueprint (your ontology). The library is the factory that turns text into structured knowledge.

CHAPTER 02

Install & set up

1 · Install the library

Install straight from source control — the package name plus the extras you need (they're additive):

```
pip install "graphrag-stage1 @ git+https://github.com/<you>/<repo>"
```

EXTRA	GIVES YOU
`graphrag-stage1` (no extra)	Stage 1 + Stage 2 (facts & frames). Only dependency is `requests`.
`[ontology]`	+ Stage 3 ontology mapping & the Neptune/SHACL gate (rdflib, pyshacl)
`[grounding]`	+ entity grounding to real OBO term IDs (oaklib). **Pair this with `[ontology]`** — see Chapter 06; without it Stage 3 types almost nothing.
`[validation]`	+ `validate()` output checking against the shipped JSON Schemas
`[anthropic]`	+ built-in Claude client
`[openai]`	+ built-in OpenAI client

Most users want everything:

```
pip install "graphrag-stage1[ontology,grounding,validation,anthropic] @ git+https://github.com/<you
```

> THE ONTOLOGIES COME WITH IT —
>
> The foundational stack — IAO, CCO (which also supplies BFO), RO, and the alignment + SHACL shapes — ships **inside the wheel**. There is nothing to download, clone, or place on disk: `[ontology]` gives you a working Stage 3 out of the box. The only ontology you provide is your own *domain* file (Chapter 06).

2 · Give it access to your model

For the built-in clients, set the standard environment variable:

```
export ANTHROPIC_API_KEY=sk-...      # or OPENAI_API_KEY=sk-...
```

3 · Turn your PDF into text

The library reads text. A few lines with PyMuPDF handle most PDFs:

```
pip install pymupdf
```

```
import fitz  # PyMuPDF
text = "\n\n".join(page.get_text() for page in fitz.open("paper.pdf"))
```

> ON AWS —
>
> Use **Amazon Textract** for the PDF→text step. For scientific papers with complex layout, **GROBID** gives cleaner results.

4 · (Optional) Place your ontology

For Stage 3, have your domain ontology file ready (Chapter 6). Skip this to get just facts.

CHAPTER 03

Your first run

The entire program is a few lines. Here it is, then a walk-through.

```
from graphrag_stage1 import AnthropicClient, analyze_paper, validate

text = open("paper.txt", encoding="utf-8").read()

results = analyze_paper(
    text,
    client=AnthropicClient(),          # your model
    max_concurrency=16,                # speed dial
    domain_ontology="my_domain.owl",   # omit this line to skip Stage 3
)

for r in results:                      # one entry per paragraph, in order
    if "error" in r:                   # a failed paragraph never stops the run
        continue
    for fact in r["stage1"]["statements"]:
        print(fact["facets"]["relation"], "|", fact["text"])
```

› WHAT EACH LINE DOES

- `AnthropicClient()` — the model. Reads `ANTHROPIC_API_KEY`. Swap for `OpenAIClient()`, `OllamaClient()`, or your own (Chapter 5).
- `analyze_paper(text, …)` — splits the text into paragraphs and runs all stages in parallel.
- `max_concurrency=16` — how many model calls run at once (Chapter 8).
- `domain_ontology=…` — loads the core ontologies plus your domain file and runs Stage 3. Leave it out for Stage 1 + 2 only.
- `results` — a list, one item per paragraph, in the same order as the paper.

> WHAT YOU'LL SEE –
>
> Lines like `CAUSATION | The fused sensor data reduced positioning error by 35 percent.` — each extracted fact and its relationship type.

CHAPTER 04

Reading the results

`results` is a list — one entry per paragraph. Each entry has `stage1`, `stage2`, and (if you passed an ontology) `stage3`.

› THE MOST USEFUL PART: THE FACTS

```
fact = results[0]["stage1"]["statements"][0]

fact["text"]                # "The fused sensor data reduced positioning error by 35 percent."
fact["facets"]["relation"]  # "CAUSATION"
fact["facets"]["modality"]  # "OBSERVED"   (fact vs. prediction vs. hypothesis)
fact["facets"]["polarity"]  # "POSITIVE"   (or "NEGATED")
fact["provenance"]          # where it came from: char span, page, verbatim source text
fact["confidence"]["overall"]   # a number (see the note in Chapter 10)
```

› WHAT EVERY FIELD TELLS YOU

YOU WANT…	READ THIS
The fact, in plain words	`fact["text"]`
Its labels (cause, measurement, prediction…)	`fact["facets"]`
Where it came from (the citation)	`fact["provenance"]`
Reasoning links between facts	`results[i]["stage1"]["relations"]`
The verified, field-by-field version	`results[i]["stage2"]["frames"]`
The knowledge-graph mapping	`results[i]["stage3"]`

› CHECK THE OUTPUT IS WELL-FORMED

```
from graphrag_stage1 import validate
validate(results[0]["stage1"])   # returns it, or raises if the shape is wrong
validate(results[0]["stage2"])
```

FAILED PARAGRAPHS —

If a paragraph couldn't be processed, its entry is `{"error": "...", "paragraph_id": "..."}` instead — check for `"error"` before reading `"stage1"`. One failure never stops the rest of the paper.

CHAPTER 05

Choose your model

The library works with any model that can return JSON. Built-ins cover the common cases with zero code; anything else is a tiny adapter.

› BUILT-IN (NO CODE)

```
from graphrag_stage1 import AnthropicClient, OpenAIClient

AnthropicClient()                              # default Claude, reads ANTHROPIC_API_KEY
AnthropicClient(model="claude-sonnet-5")       # pick any model
OpenAIClient(model="gpt-4o-mini")              # reads OPENAI_API_KEY
```

› ANY OPENAI-COMPATIBLE ENDPOINT (VLLM, GROQ, TOGETHER, LOCAL SERVERS)

```
import openai
from graphrag_stage1 import OpenAIClient
client = OpenAIClient(sdk=openai.OpenAI(base_url="http://my-server/v1", api_key="..."))
```

› LOCAL, OFFLINE (OLLAMA)

```
from graphrag_stage1 import OllamaClient
client = OllamaClient(model="qwen3:8b")   # talks to a local Ollama server
```

› YOUR OWN MODEL — IMPLEMENT ONE METHOD

```
class MyClient:
    model_id = "my-provider/my-model"      # recorded in every result's provenance
    def complete(self, prompt, schema, *, stronger=False):
        # call your model; force JSON matching `schema`; return the dict
        ...
```

TWO THINGS TO KNOW —

Speed & quality depend on the model you choose — a fast model (Haiku, gpt-4o-mini) for throughput, a stronger one for accuracy. And your `complete` is called from many threads at once, so it must be **thread-safe** (the built-ins already are).

SEE ALSO —

A full runnable cookbook for every provider — Claude, GPT, Azure, Bedrock, Gemini, vLLM, Ollama — with pros/cons is in `examples/model_clients.py` and `docs/CHOOSING_A_MODEL.md`.

CHAPTER 06

Use your own ontology

*The foundational ontologies ship **inside the package** and load themselves. You supply exactly one thing: the domain ontology that matches your papers.*

› WHERE FILES LIVE

```
graphrag_stage1/ontologies/        ← SHIPPED IN THE WHEEL, nothing to fetch
├── iao.owl                          always loaded
├── CommonCoreOntologiesMerged.ttl   always loaded (also supplies BFO)
├── Relations/ro.owl                 relation vocabulary
└── Alignment/                       pipeline→ontology bridge + SHACL shapes

<anywhere you like>/my_domain.owl   ← YOUR file, passed explicitly
```

WHY BFO HAS NO FILE OF ITS OWN —

CCO and IAO are both built on BFO and republish its classes under the canonical `obo/BFO_*` IRIs, so BFO arrives with them. A separate BFO file is worse than useless — a stale one names the same concepts under *different* IRIs, which makes every upper-ontology term ambiguous.

› PLUG IN YOUR DOMAIN ONTOLOGY

```
# The core stack comes from the installed package. Just name your domain file:
results = analyze_paper(text, client=c, domain_ontology="/path/to/my_domain.owl")
```

You can point at a *folder* of ontology files, not just one. To load the core stack from somewhere else (a pinned or patched copy), pass `ontology_root="/path/to/your/ontologies"` — otherwise leave it alone and the packaged copy is used.

WHY THE DOMAIN ONTOLOGY ISN'T BUNDLED —

It is the one module that changes per scientific domain — your choice, not ours — and it can dwarf the core: the whole foundational stack is ~3.7 MB, while UBERON alone is 48 MB.

› WHAT MAKES A GOOD DOMAIN ONTOLOGY

- A proper OWL/RDF file: `.owl`, `.ttl`, `.rdf`, `.nt`, `.n3`, or `.jsonld`.
- **Human-readable labels** on its classes and properties — that's what your facts get matched against.
- Ideally built on BFO (like CCO), so it stacks cleanly onto the core.

› ALSO PASS A GROUNDER — OR ALMOST NOTHING GETS TYPED

THIS IS THE MOST COMMON WAY TO GET A DISAPPOINTING GRAPH —

Loading a domain ontology is only half the job. Your papers say *"dendrobine"* and *"COX-2"*; your ontology has classes with formal labels. When an entity isn't already a class in the loaded file, Stage 3 can only generalise it to a *true* ancestor — and the honest true ancestor of an unrecognised thing is `BFO:entity`, the root of the whole ontology. Everything "maps", and nothing means anything.

A grounder resolves mentions to real ontology term IDs **lexically** — never by asking a model to recall an ID, because models hallucinate CURIEs and a lexical match cannot. Measured on the 4 most interaction-dense paragraphs of a PMC paper, against INO:

STAGE 3 RUN	ENTITIES GIVEN A REAL DOMAIN TYPE
ontology only, no grounder	**0 of 10** — every entity fell back to `BFO:entity`
ontology + `OakGrounder()`	**7 of 21 (33%)** — real `CHEBI`, `GO`, `MONDO`, `PR`, `PW` classes

```
from graphrag_stage1.grounding import OakGrounder

results = analyze_paper(
    text, client=c,
    domain_ontology="ontologies/Domain/my_domain.owl",
    grounder=OakGrounder(cache_path=".grounding_cache.json"),  # ← add this
)
```

The grounder searches every OBO ontology at once over HTTP (no local database to download) and caches to disk, so the network is hit **once per distinct mention across all runs**. It is optional and fails soft: if `oaklib` isn't installed or the service is unreachable, every lookup returns "not found" and you simply fall back to the in-graph typing — grounding never crashes a run.

> **RULE OF THUMB —**
> The domain ontology supplies the *frame* vocabulary (what kind of claim this is). The grounder supplies the *entity* vocabulary (what the nouns actually are). You want both; they are not substitutes.

> **PLUG-AND-PLAY —**
> There are no hardcoded domain terms in the code — it reads the labels inside whatever ontology you load. Swap the file, and the mapping targets change with it. Terms it can't map confidently are logged, never guessed.

CHAPTER 07

Produce the Neptune file

Stage 3 gives you the mapping. One more call turns it into the N-Quads file you upload to a graph database.

```
from graphrag_stage1.stage3_neptune_export import export_neptune_nquads

meta = export_neptune_nquads(
    results[0]["stage3"],
    destination="paragraph0.nq",                    # ← the file to upload
    quarantine_destination="rejected.ttl",          # optional: what was filtered out
    report_destination="report.txt",                # optional: the validation report
)
print(meta["validated_triple_count"], "triples ->", meta["destination"])
```

› WHAT HAPPENS IN THAT CALL

- The mapped facts are turned into RDF triples, each document in its own *named graph*.
- A **SHACL validation** gate checks every triple. Clean ones go to the `.nq` file; bad ones are *quarantined* to a separate file and never uploaded.
- You get back a summary: how many triples passed, how many were quarantined.

› THEN LOAD IT

```
Upload  *.nq  →  S3  →  run the Neptune bulk loader
```

> **NEPTUNE DOES NOT REASON FOR YOU —**
>
> Loading the file into Neptune stores and lets you query the facts — it does **not** automatically infer new ones from the ontology. OWL reasoning (deriving implied facts) is a separate step you add afterward if you want it. It is not part of this library.

CHAPTER 08

Make it fast

Which dial matters depends entirely on whether your model endpoint can scale.

> **WHAT A REAL RUN LOOKS LIKE —**
>
> All three stages, from a `pip install`ed wheel, on the 4 most interaction-dense paragraphs of a real PMC paper (qwen3:8b local, defaults, user-supplied INO domain ontology):
>
> **36 facts → 36 frames → RDF in 105 s** (26 s/paragraph), 0 errors, 4/4 paragraphs passing the Stage 1 + Stage 2 JSON Schemas, and 8 of 22 entities given real domain types (`MONDO`, `PW`, `CL`, `GO`, `PR`, `CHEBI`) rather than `BFO:entity` fallbacks.

› ON A HOSTED API: TURN UP THE CONCURRENCY

Paragraphs are independent, so the library runs them in parallel. On an endpoint that scales, more concurrent calls really do finish sooner:

> *time ≈ (number of model calls × latency per call) ÷ max_concurrency*

```
analyze_paper(text, client=c, max_concurrency=16,   # total calls at once
              stage2_concurrency=6)                  # per-paragraph fan-out
```

Set `max_concurrency` to your provider's safe concurrent-request budget. Too low is slow; too high gets you rate-limited. A ~50-paragraph paper lands in **about a minute**.

> **"SAFE BUDGET" IS NOT A SUGGESTION —**
>
> This only works for concurrency you are actually entitled to. Measured on a free Ollama Cloud tier: `max_concurrency=4` completed cleanly, while `12` returned HTTP 429 on **7 of 12 paragraphs** — and because each failure waits out the full backoff first, the run was also **3× slower per surviving paragraph**. Past your quota the formula above inverts: more concurrency, less throughput, fewer facts.

› ON A SINGLE LOCAL GPU: THAT FORMULA STOPS WORKING

Raising `max_concurrency` past a few does nothing. The GPU is not queueing your calls politely — it is already **saturated at 97–98% utilisation**, and extra concurrent requests simply time-slice the same silicon. You must make the model do *less work*, not give it more work at once.

Where the time actually goes, measured on one paragraph (qwen3, laptop RTX 5080):

CALL	READING THE PROMPT	WRITING THE ANSWER
Stage 1 (decompose)	0.1 s — 749 tokens	2.5 s — 117 tokens
Stage 2 (one frame)	0.5 s — 2,184 tokens	**18.7 s — 819 tokens**

Reading is nearly free; writing is the entire cost. A Stage 2 frame writes about seven times as many tokens as a Stage 1 call, and Stage 2 runs once per fact rather than once per paragraph. That is your bottleneck.

› THE TRAP: A SMALLER MODEL FOR STAGE 2 MAKES THINGS WORSE

This is the obvious move, and it backfires. Every Stage 2 frame is checked against the source text, and a frame that fails the check is **reprocessed — a second full call**. A weaker model fails more often, so it buys a cheaper call and pays for two. Measured on the same 43 facts, changing only the Stage 2 model:

STAGE 2 MODEL	FACTS ACCEPTED	TIME
`qwen3:4b` (smaller, "faster")	4 of 43 (9%)	268 s
`qwen3:8b`	**25 of 43 (58%)**	**243 s**

The bigger model was **faster and six times better**. On this pipeline, model quality is a *speed* feature.

› WHAT ACTUALLY WORKS: MAKE FEWER CALLS

LEVER	HOW	EFFECT
Batch the facts (biggest win)	`stage2_batch_size=5` — **now the default**	Frames 5 facts in one call instead of 5 calls. Measured **1.75× faster and more accurate** — seeing its sibling facts helps the model stay grounded, which kills the retries. On by default since 0.2; set `STAGE2_BATCH_SIZE=0` to go back. Don't go past ~10.
Size the context window (see below)	`OllamaClient(num_ctx=6144)` — **now the default**	Batching needs room to write 5 frames. Too small and every batch truncates; too large and it thrashes your VRAM. Worth **10×** — the single biggest number in this chapter.
Skip the recall pass when it can't help	automatic since 0.2	The recall pass now runs only when the deterministic floor says a clause went missing, saving ~1 call per paragraph. `STAGE1_ALWAYS_RECALL=1` restores it everywhere.
Drop the retry pass	`STAGE2_MAX_RETRIES=0`	Each reprocessed fact is a second full call. Costs you the auto-repair pass.
Feed it fewer paragraphs	automatic since 0.2 (`gate_noise=True`)	Keywords lines, supplementary-material DOIs and publisher notices are dropped before they cost a call. Narrow by design: ~5% of a real paper. It won't guess — figure captions and methods lines are content, so they go to the model.

> **DON'T BOTHER —**
> Shrinking or batching prompts. Reading the prompt is only ~3% of a Stage 2 call, so there is nothing there to win.

› THE ONE NUMBER WORTH 10×: `NUM_CTX` × `OLLAMA_NUM_PARALLEL`

Ollama gives every model a 4096-token window by default, and allocates that window *per parallel slot*. Both halves of that sentence bite:

- **Too small** — a batch of 5 needs ~5×800 output tokens. Against a 4096 window every batch silently truncates into invalid JSON, falls back to the per-statement path, and the run ends up *slower than not batching at all*.

- **Too large** — with `OLLAMA_NUM_PARALLEL=4`, a `num_ctx` of 8192 reserves **32K** of KV cache, which thrashes VRAM next to a 7.4 GB model on a 16 GB card.

Measured on 4 real paragraphs (qwen3:8b, RTX 5080 16 GB, `OLLAMA_NUM_PARALLEL=4`, batch 5), changing **only** this value:

NUM_CTX	WALL	OUTCOME
8192	**1026 s**	1 paragraph failed outright (thrashing)
6144 (default)	**101 s**	0 failures

If you raise `num_ctx`, lower `OLLAMA_NUM_PARALLEL` to match your VRAM — otherwise the headroom you bought for a bigger batch is exactly what thrashes.

› WHAT A REAL PAPER ACTUALLY COSTS

With the 0.2 defaults, on 4 genuine 70–75-word paragraphs from a PMC paper:

SETUP	PER PARAGRAPH	80-PARAGRAPH PAPER	ERRORS
Local qwen3:8b, `max_concurrency=4`	**25 s**	**~33 min**	0
Ollama Cloud `gpt-oss:120b-cloud`, `max_concurrency=4`	55 s	~73 min	0
Ollama Cloud, `max_concurrency=12`	154 s	—	**7 of 12 lost to 429s**

> **IF A PAPER IS TAKING HOURS –**
> Check these two settings before blaming your hardware. With batching off and a 4096 window — the pre-0.2 defaults — that same 80-paragraph paper took **~8 hours**. The defaults were the bottleneck, not the GPU.

› A BIGGER MODEL IS NOT A FASTER ONE

This surprises people, so it is worth stating plainly: **your laptop is probably faster than the cloud.** A 120B cloud model is 2.2× slower *per paragraph* than an 8B on your own GPU, and reasoning models (`nemotron-*`) are slower still because they spend tokens thinking before they answer. What a big model buys is a better graph — on the run above, `gpt-oss:120b-cloud` produced 7 fully-mapped frames against local's 3.

Speed comes from **parallel capacity**, which is a property of your quota or your hardware — not of the model. And you only get the concurrency you are entitled to: on a free Ollama Cloud tier, `max_concurrency=4` runs clean while 12 returns HTTP 429 on more than half the paper and each failure burns the full backoff first. Set `max_concurrency` *to* your allowance. Past it you buy waiting and data loss, not throughput.

CHAPTER 09

Recipes & cheat sheet

Common tasks, ready to copy.

› GET JUST THE FACTS AS PLAIN TEXT

```
facts = [s["text"] for r in results if "stage1" in r
                   for s in r["stage1"]["statements"]]
```

› KEEP ONLY CAUSAL FACTS

```
causal = [s for r in results if "stage1" in r
             for s in r["stage1"]["statements"]
             if s["facets"]["relation"] in ("CAUSATION", "MECHANISM")]
```

› KEEP ONLY HIGH-CONFIDENCE FACTS

```
strong = [s for r in results if "stage1" in r
             for s in r["stage1"]["statements"]
             if s["confidence"]["overall"] >= 0.8]
```

› PROCESS A WHOLE FOLDER, STREAMING PROGRESS

```
import glob
from graphrag_stage1 import run_paper, split_paragraphs

def on_done(i, r):
    print("finished paragraph", i)

for path in glob.glob("papers/*.txt"):
    paras = split_paragraphs(open(path, encoding="utf-8").read())
    results = run_paper(paras, client=c, max_concurrency=16, on_result=on_done)
```

› THE FULL QUICK-REFERENCE

TO GET…	CALL / READ
Facts + frames (+ graph) for a paper	`analyze_paper(text, client=c[, domain_ontology=…])`
Facts for one paragraph	`process_paragraph(text, client=c)`
The Neptune file	`export_neptune_nquads(r["stage3"], "out.nq")`
The facts	`r["stage1"]["statements"]`
The reasoning links	`r["stage1"]["relations"]`
Validate an output	`validate(r["stage1"])`

CHAPTER 10

Troubleshooting & FAQ

› "NO MODULE NAMED GRAPHRAG_STAGE1"

It isn't installed in the environment you're running. Run `pip install …` in the same Python/venv, or install it into your container image.

› "IT WON'T TAKE MY PDF"

Correct — it reads **text**, not PDFs. Convert first with PyMuPDF/Textract/GROBID (Chapter 2), then pass the text.

› STAGE 3 GIVES AN EMPTY OR THIN GRAPH

Usually a **domain-ontology mismatch** — the ontology doesn't cover your paper's subject, so terms fail closed (by design). Use an ontology that matches your field, with good labels.

› RATE-LIMIT / THROTTLING ERRORS

Lower `max_concurrency` to your provider's limit. That single cap controls total calls in flight.

› IT'S SLOW

You're likely on a single local GPU (which serializes) or `max_concurrency` is low. Use a hosted/scalable endpoint and raise the cap (Chapter 8).

› STRUCTURED-OUTPUT / JSON ERRORS FROM MY CUSTOM MODEL

Your `complete` must return a dict matching the given `schema`. Use your provider's tool-calling / JSON-schema mode (the built-in clients show the pattern). Very small local models sometimes can't do reliable structured output.

> **ABOUT THE CONFIDENCE SCORES —**
>
> Trust the **structure, the exact numbers, and the provenance** today — those are deterministic guarantees. The numeric **confidence values are not yet calibrated** (the evaluation set is still small), so don't gate important decisions purely on them.

> **WHERE TO GO NEXT —**
>
> `DOCUMENTATION.md` — the complete field-by-field API reference. `INTEGRATION.md` — integrating into another system. `DESIGN.md` — how and why it works inside.

CHAPTER 11

Jupyter Notebook Workflow

A complete, runnable notebook is at `examples/jupyter_usage_demo.ipynb`. Open it, run the cells, and adapt to your papers.

› OPEN IT

```
jupyter lab examples/jupyter_usage_demo.ipynb
# or
jupyter notebook examples/jupyter_usage_demo.ipynb
```

› WHAT'S INSIDE (9 SECTIONS)

SECTION	COVERS
1. Installation	`%pip install graphrag-stage1` + optional extras (`[ontology]` , `[anthropic]` , `[openai]` , `[dev]`)
2. Imports & LLM Client	All imports; choose Anthropic, OpenAI, Ollama, or custom `LLMClient`
3. Stage 1 Only	`process_paragraph()` → Knowledge Artifact Graph (propositions, discourse relations, coreference)
4. Stage 1 → 2	`stage2_pipeline()` → Semantic frames with typed slots, grounding checks, confidence
5. Stage 3 (Ontology)	`OntologyManager` + `run_pipeline(ontology=...)` → RDF/JSON-LD + Turtle export
6. Full Paper	`analyze_paper()` / `run_paper()` for multi-paragraph parallel processing
7. Validation	`validate()` against versioned JSON Schemas
8. Exploration & Debugging	Inspect provenance, mapping audits, enable debug logging
9. Patterns & Tips	Custom clients, checkpointing, error handling, performance tuning, output formats

› MINIMAL RUNNABLE EXAMPLE (3 CELLS)

```
# Cell 1: Install
%pip install graphrag-stage1

# Cell 2: Imports + client
from graphrag_stage1 import run_pipeline, OllamaClient
client = OllamaClient()  # or AnthropicClient(), OpenAIClient()

# Cell 3: Run pipeline
text = "The fused sensor data reduced positioning error by 42% because it provides a more complete s
result = run_pipeline(text, client=client, paragraph_id="demo:p1")

# Cell 4: Inspect outputs
kag = result["stage1"]      # Knowledge Artifact Graph
frames = result["stage2"]   # Validated semantic frames

print(f"Propositions: {len(kag['propositions'])}")
print(f"Frames: {len(frames['frames'])}")
for f in frames["frames"]:
    print(f"  {f['frame_type']}: confidence={f['confidence']:.2f}")
```

› WITH STAGE 3 (ONTOLOGY MAPPING)

```
# Requires: %pip install "graphrag-stage1[ontology]"
# And: ontologies/ directory with core + your domain ontology

from graphrag_stage1.ontology_manager import OntologyManager

ontology = OntologyManager("ontologies").load_all(domain_ontology="Domain/your_domain.owl")

result = run_pipeline(
    text, client=client, paragraph_id="demo:p1",
    ontology=ontology  # <-- triggers Stage 3
)

stage3 = result["stage3"]
print(f"Mapped frames: {len(stage3['mapped_frames'])}")
print(f"Ontology individuals: {len(stage3['ontology_individuals'])}")

# Export for Neptune / GraphDB
from graphrag_stage1.stage3_jsonld import build_jsonld
jsonld = build_jsonld(stage3)
import json
with open("output.jsonld", "w") as f:
    json.dump(jsonld, f, indent=2)
```

› KEY PATTERNS FOR NOTEBOOK WORKFLOWS

PATTERN	CODE SNIPPET
Checkpointing	`run_paper(paras, client=c, on_result=lambda i, r: save_checkpoint(i, r))`
Error handling	`if "error" in r: log(r["error"]); continue`
Provenance citation	`prop["provenance"]["source_statement"]` gives verbatim text span
Debug LLM calls	`logging.getLogger("graphrag_stage1.llm").setLevel(logging.DEBUG)`
Batch + aggregate	Collect all `stage3["mapped_frames"]` → merge → single KG

› ENVIRONMENT VARIABLES FOR LOCAL DEVELOPMENT

```
# .env or shell export
export STAGE1_MODEL="qwen3:8b"
export STAGE2_MODEL="qwen3:8b"
export STAGE2_STRONGER_MODEL="qwen3:14b"
export OLLAMA_URL="http://localhost:11434/api/generate"

# Or set in notebook:
import os
os.environ["STAGE1_MODEL"] = "qwen3:8b"
os.environ["STAGE2_MODEL"] = "qwen3:8b"
```

> **FULL REFERENCE —**
>
> `DOCUMENTATION.md` §21 has the complete notebook guide with all sections.
